

Module 19 Week 22: Generative AI application – Part 2

Introduction to LLM Selection and AI Agents

- Large Language Models vary in performance, size, cost, and specialisation, requiring careful selection based on project needs and infrastructure.

Factor	Considerations
Performance	Required level of accuracy and capability for your specific task
Size and speed	Can infrastructure support large models or is a lightweight model needed?
Specialisation	Need for general-purpose versus domain-specific knowledge
Ethical considerations	Privacy implications and transparency of model development
Deployment environment	Intended deployment: Cloud, on-premises or edge devices
Cost	Budget for model usage, licensing or fine-tuning
Customisability	Need for fine-tuning to tailor the model to your use case

- AI agents combine perception, decision-making, and action modules to perform tasks across domains like customer service, healthcare, and education.



- LLMs serve as the core of AI agents, enabling natural language understanding, complex reasoning, and human-like interaction capabilities.
- Integration challenges include managing context, ensuring consistent behavior, and optimising performance for scalable and reliable agent deployment.
- Ethical considerations such as privacy, bias, and transparency must guide the development of responsible and trustworthy AI-powered systems.

LLM Performance and Benchmarks

- The expanding variety of large language models necessitates standardised benchmarks to evaluate and compare performance across different tasks, capabilities, and application domains.
- BIG-Bench Hard tests general language understanding through over 200 diverse tasks, making it useful for assessing versatility, though results can vary widely across categories.



- MBPP evaluates LLMs on beginner-level Python problems, providing insight into programming support capabilities, but it does not reflect advanced coding performance.

- Task description
 - Write a function `is_palindrome(s)` that checks whether a given string `s` is a palindrome.
- Solution


```
def is_palindrome(s):
    return s == s[::-1]
```
- Test cases


```
assert is_palindrome("racecar") == True
assert is_palindrome("hello") == False
assert is_palindrome("madam") == True
```

- MMLU assesses multilingual language understanding and five-shot learning across 57 tasks, offering a strong indication of how models adapt with minimal training data.
- HumanEval and TriviaQA focus on real-world coding accuracy and one-shot knowledge retrieval respectively, highlighting practical effectiveness in software development and question answering scenarios.

Question:	Question: Which Apollo 11 astronaut took a historic first step onto the Moon?	• Tests knowledge retrieval with only one example • Covers 100,000 varied-difficulty Q&A pairs • Evaluates quick adaptation to question-answering tasks
Answer:	Answer: Neil Armstrong	
1-Shot Learning:	The Apollo 11 astronaut was Neil Armstrong.	

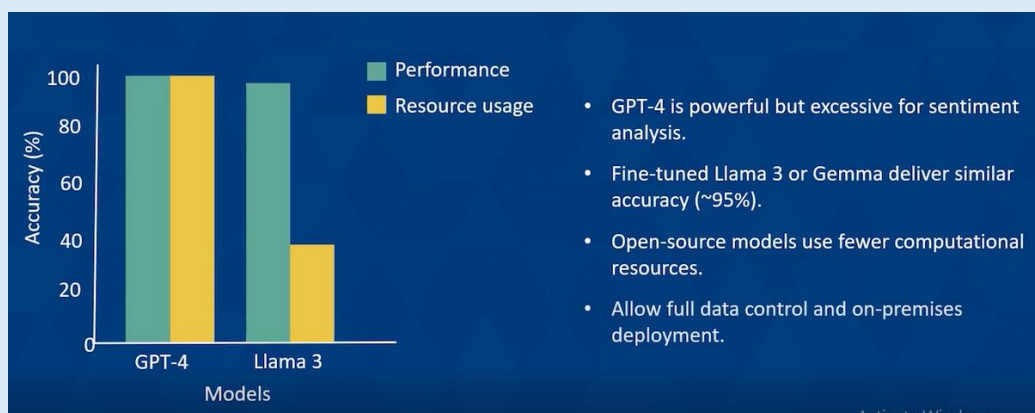
Evaluating LLM Requirements for Specific Use Case

- The choice of language model should be guided by the specific task, with smaller, fine-tuned open-source models often outperforming larger ones in efficiency and relevance.

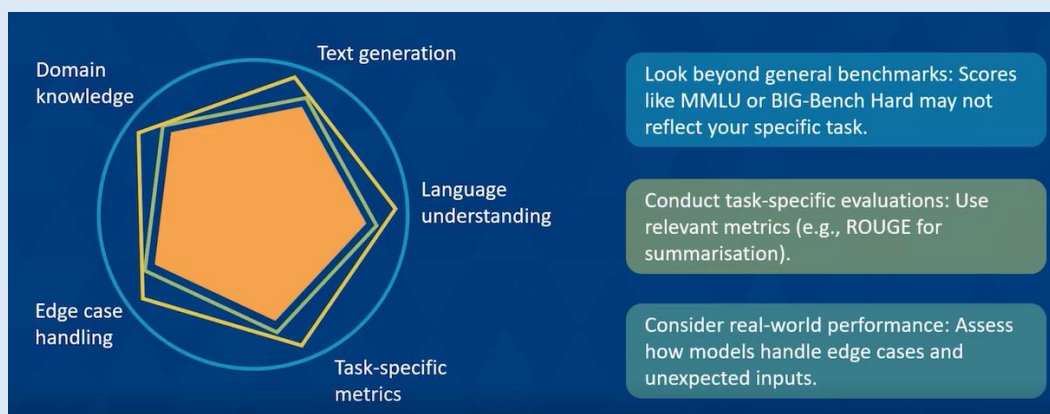
- Key evaluation criteria include task specificity, required performance level, infrastructure and budget limitations, and the need for control over data and deployment.

- Task specificity: Define whether the task is narrow (e.g., sentiment analysis) or broad in scope.
- Performance metrics: Determine required accuracy and fluency levels
- Resource constraints: Assess infrastructure, budget, and time availability
- Data privacy and control: Decide how critical full control over data and model is

- Real-world examples show that fine-tuned models like LLaMA 3 or Gemma can match GPT-4 in performance for tasks such as sentiment analysis while being more resource-efficient.



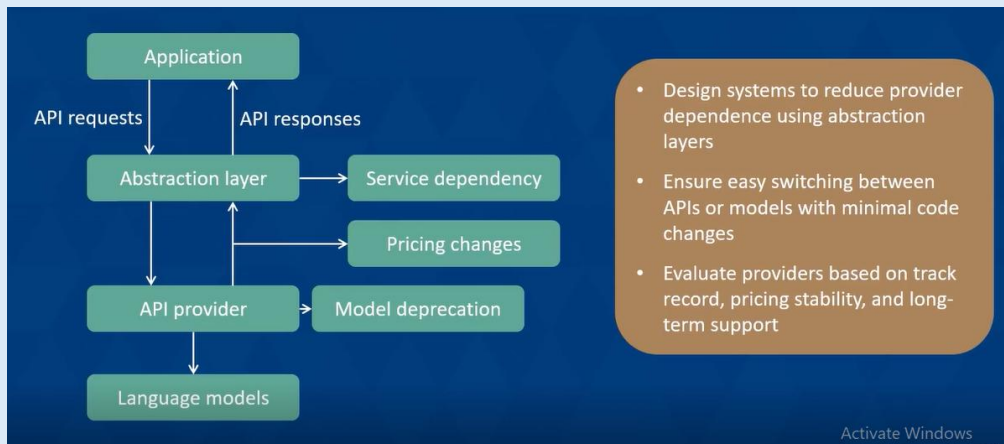
- General benchmarks like MMLU or BIG-Bench Hard are informative but may not reflect real-world performance, making task-specific evaluations essential for accurate model selection.



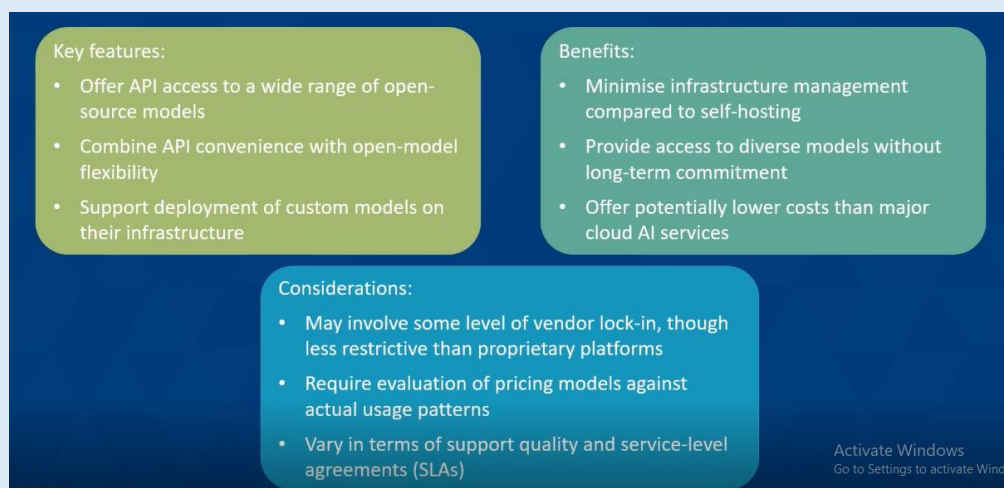
- Deployment strategies—whether on-premises for control and privacy or cloud-based for scalability—must align with team expertise, latency requirements, and long-term maintenance capacity.

LLM Deployment Considerations

- API-based deployment offers rapid access to state-of-the-art models with minimal infrastructure setup, but introduces risks related to vendor dependency, pricing volatility, and potential service discontinuation.



- Private deployment of open-source models ensures maximum control, customisation, and data privacy, though it demands significant computational resources, in-house expertise, and higher initial investment.
- GPU-as-a-service platforms provide a flexible middle ground by enabling access to open-source models with reduced infrastructure overhead while posing lower vendor lock-in risk than proprietary APIs.



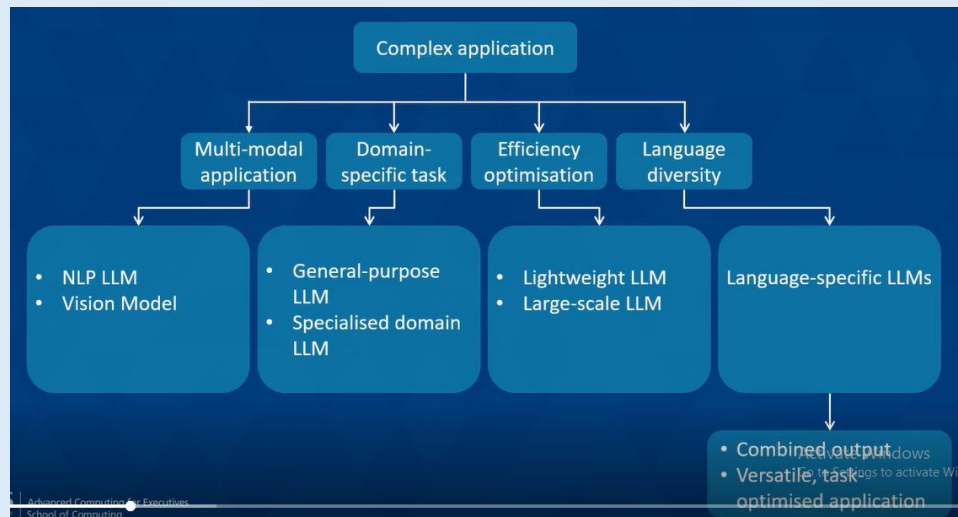
- Hybrid deployment strategies enable organisations to balance speed, cost, privacy, and reliability by combining public APIs, private infrastructure, and third-party GPU services in a modular architecture.

- **Benefits:**
 - Use API services for rapid prototyping and initial deployment
 - Transition critical or high-volume components to private deployment
 - Retain API access as a backup or for specific features
- **Why this approach works:**
 - Start quickly with APIs while gaining private deployment expertise
 - Optimise costs by matching deployment type to task volume
 - Improve reliability through deployment redundancy

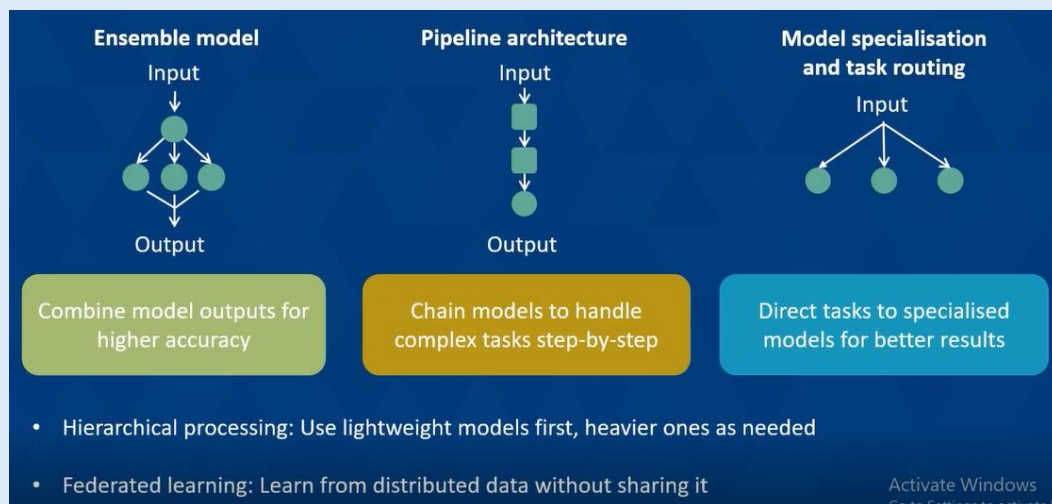
- Designing with provider independence in mind through abstraction layers, fallback mechanisms, and multi-provider strategies safeguards applications from disruptions and future-proofs long-term AI deployments.

Combining Multiple LLMs for Complex Applications

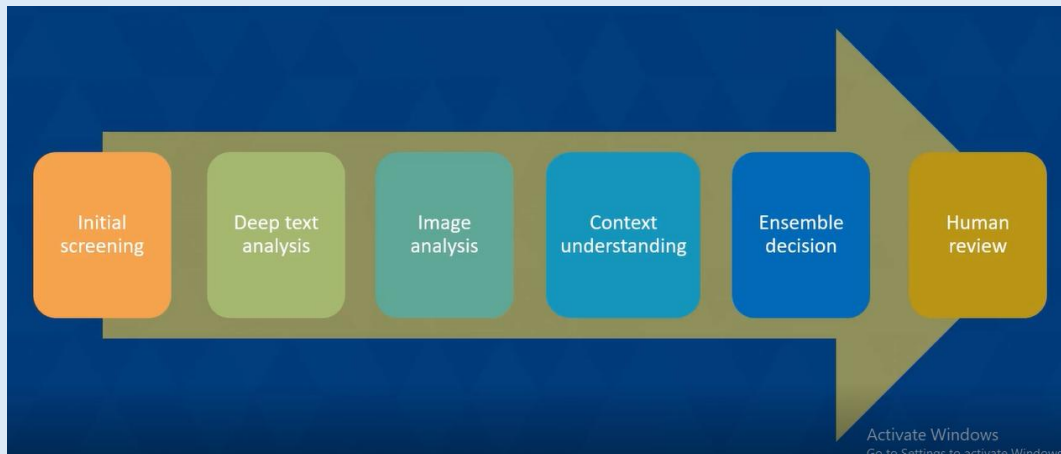
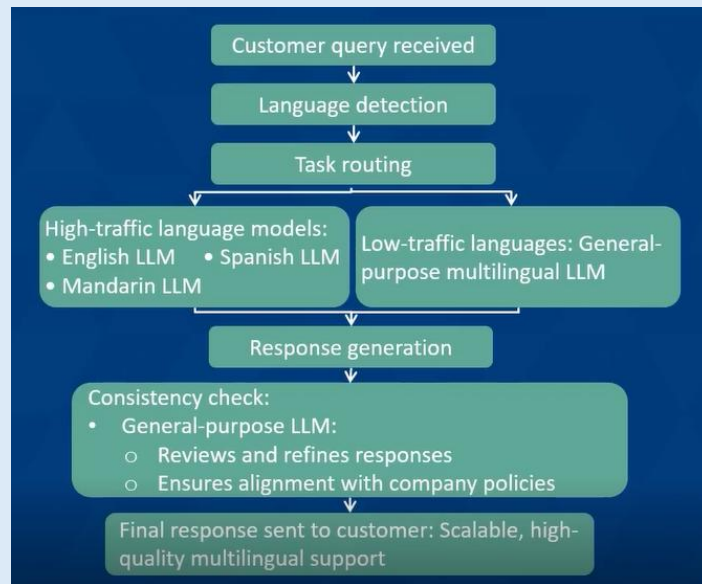
- Multi-model architectures enhance AI applications by leveraging specialised LLMs for tasks like language detection, domain-specific reasoning, and multimodal processing.



- Integration strategies include ensemble voting, pipeline chaining, task routing, and hierarchical processing to balance performance and resource efficiency.



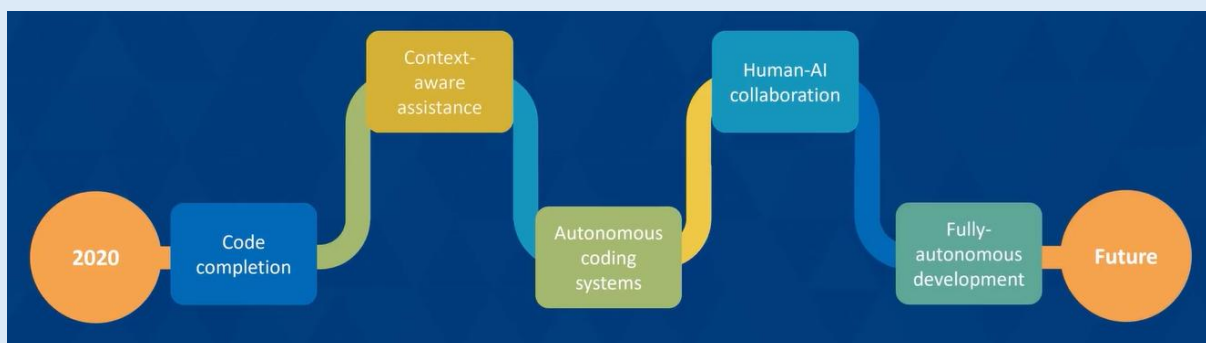
- Real-world examples such as e-commerce support and social media moderation demonstrate scalable, context-aware systems using multiple LLMs and vision models.



- Challenges include managing system complexity, ensuring consistent outputs, handling latency, and maintaining version control across diverse models.
- Modular design and reinforcement learning from human feedback support continuous improvement and adaptability in multi-model AI systems.

Introduction to AI-Powered Software Development

- AI-powered software development has progressed from basic code completion tools to context-aware assistants and is moving toward autonomous systems capable of handling entire application lifecycles.



- Tools like Devin AI promise autonomous coding, debugging, and deployment, though claims of full autonomy have sparked scepticism and driven the creation of open-source alternatives like OpenDevin.

- **Devin AI - Promises and potential:**
 - Claims autonomous coding, debugging and app building
 - Designed to learn continuously and collaborate with humans
- **Developer community - Response and Concerns:**
 - Raised doubts about the authenticity of demos
 - Criticised as potential marketing hype or premature claims

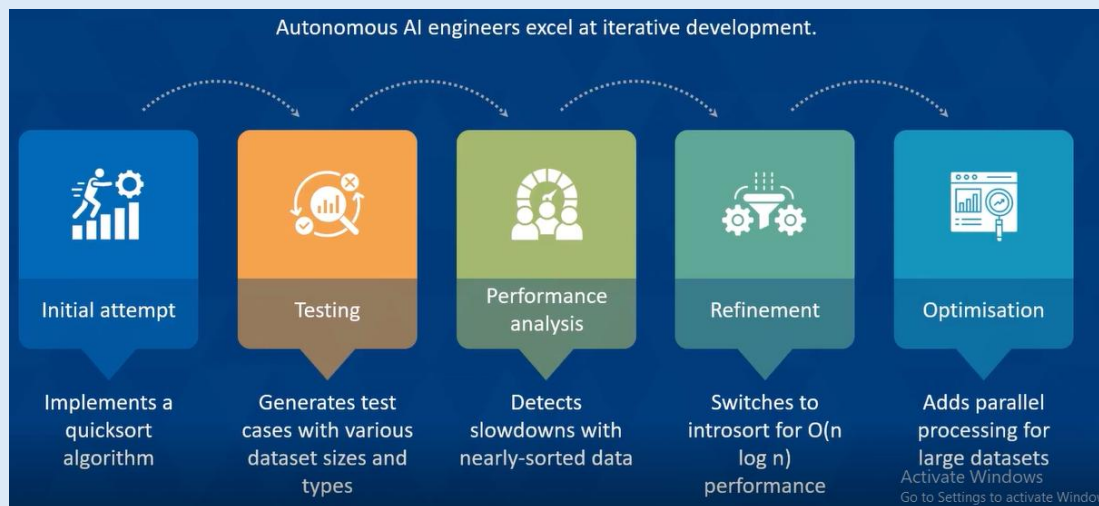
- Current AI tools already contribute significantly by generating code, detecting bugs, suggesting refactoring, and automating documentation, enabling developers to focus on higher-level design tasks.
- Key challenges include ensuring reliability, handling contextual nuances, fostering creativity, and addressing ethical concerns, all of which reinforce the need for human oversight and judgement.



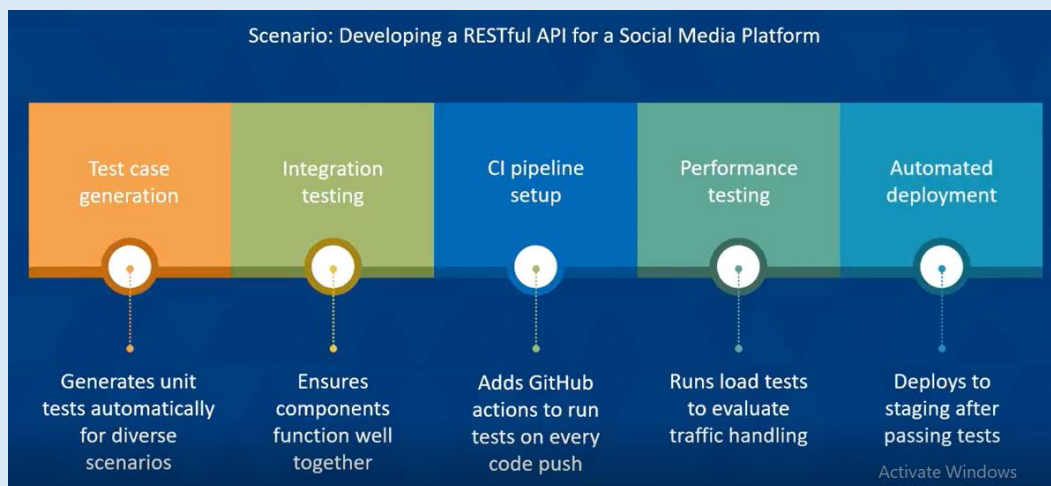
- The future of software development lies in deeper AI integration, evolving human roles, and collaborative workflows that merge AI efficiency with human creativity and problem-solving.

Future of Autonomous AI Software Engineers

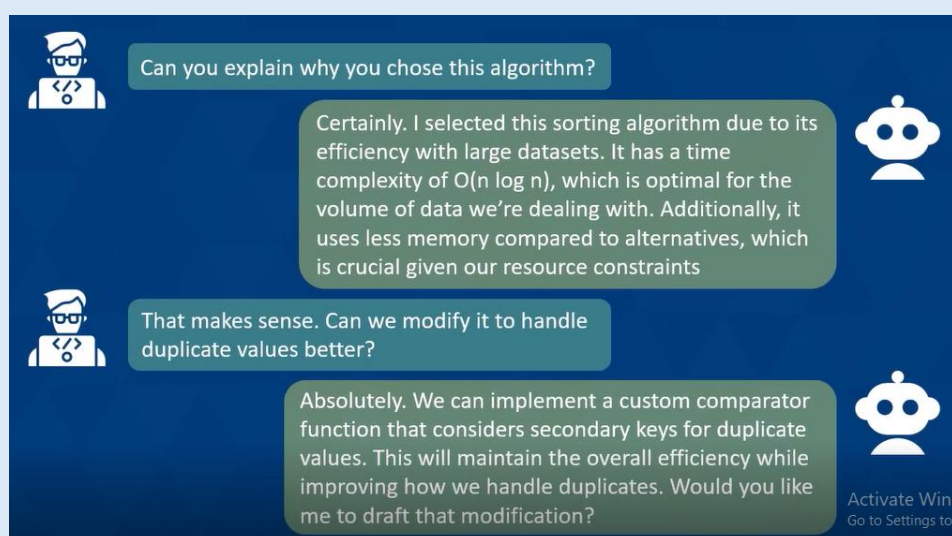
- Autonomous AI engineers demonstrate advanced capabilities such as decomposing complex problems, proposing architecture strategies, and evaluating trade-offs across components in software projects.
- Iterative development enables AI systems to test, analyse, refine, and optimise solutions based on performance metrics, illustrating a growing capacity for real-world software problem-solving.



- Integration with external systems includes tasks like secure API usage, error handling, and synchronisation, highlighting the AI's ability to consider reliability, performance, and security requirements.



- Effective collaboration between AI and human developers depends on clear communication, justifiable decisions, and the ability to explain, adapt, and revise solutions during development.



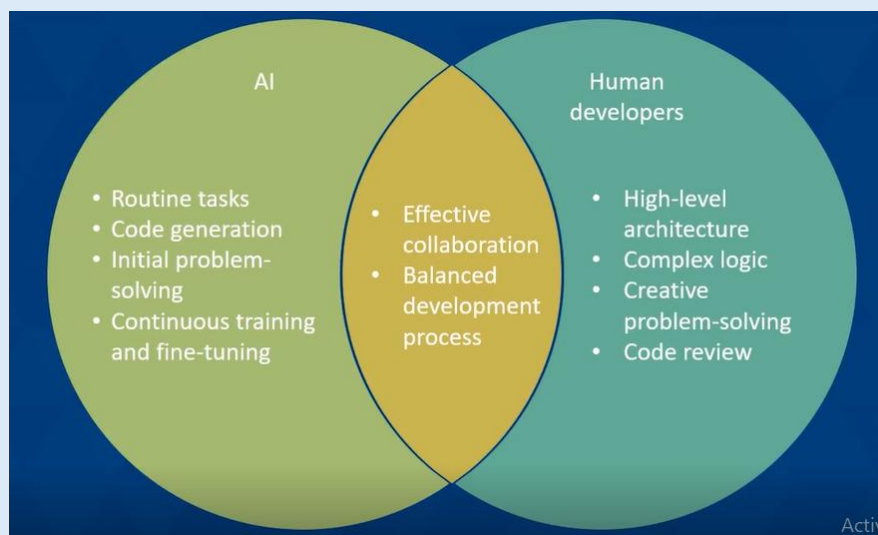
- Continuous learning allows AI systems to improve through experience, but full autonomy remains limited, reinforcing the need for human oversight in critical decision-making and complex tasks.

Pros and Cons of Autonomous AI Agents in Software Development

- Autonomous AI agents accelerate development by generating code, maintaining standards, and adapting to new technologies without downtime.
- Challenges include potential bugs, security risks, ethical concerns, and reduced developer skill due to over-reliance on AI-generated code.



- Balanced workflows combine AI for routine tasks with human oversight for architecture, complex logic, and quality assurance.



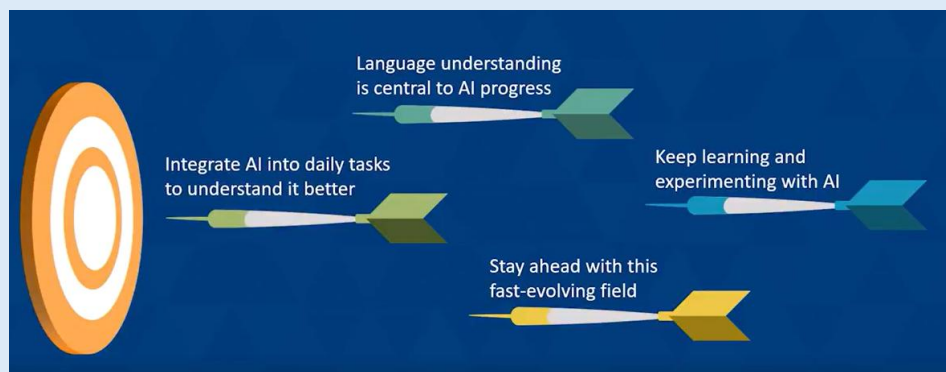
- Future development will feature deeper AI integration, enhanced collaboration tools, and evolving methodologies for human-AI synergy.

- Smarter AI: Better context understanding and problem-solving
- Deeper integration: AI embedded across the development lifecycle
- New methodologies: Practices designed for human-AI collaboration
- Ethical focus: Emphasis on responsible and fair AI use

- Ethical responsibility, continuous learning, and adaptability remain essential for developers navigating an increasingly AI-driven software landscape.

Module Summary

- AI tools enhance productivity across tasks beyond coding, helping users understand both their strengths and limitations through regular use.
- Language models form the backbone of generative AI, enabling intelligent content creation and interaction across diverse domains.
- Rapid evolution in AI demands continuous learning and experimentation to stay relevant and effective in software development.
- Completion of the course positions learners at the forefront of the AI revolution, ready to build innovative, intelligent applications.
- Creativity and problem-solving combined with AI collaboration unlock new possibilities for impactful and future-ready software solutions.



THE END